Deadlock program

```
class Paper {
}

class Pen {
}

class John extends Thread {

    Paper paper;
    Pen pen;

    John(Paper paper, Pen pen) {
        this.paper = paper;
        this.pen = pen;
    }

    public void run() {
        //John gets paper//paper object is locked
        synchronized (paper) {

           System.out.println("John tries to use(lock) Paper-Paper
avilable(not locked) - John using Paper -  Paper locked");
           // System.out.println("John: Waiting for Pen...");
           System.out.println("John tries to use(lock) pen-- Locked
by David - john is waiting to get(lock) pen...");
            synchronized (pen) {
                System.out.println("John: Pen locked");
            }
        }
    }
}

class David extends Thread {

    Paper paper;
    Pen pen;

    David(Paper paper, Pen pen) {
        this.paper = paper;
        this.pen = pen;
    }

    public void run() {
        //David gets pen object - that pen object is locked
        synchronized (pen) {
```

```
            System.out.println("David tries to use(lock) Pen - Pen
available(not locked) - David  using pen - Pen locked");
            //System.out.println("David: Waiting for Paper...");
            System.out.println("David tries to use(lock)
Paper..Locked by John-david is waiting to get Paper");
            synchronized (paper) {
                System.out.println("David: Paper locked");
            }
        }
    }
}

public class DeadlockMain {

    public static void main(String[] args) {

        Paper paper = new Paper();
        Pen pen = new Pen();

        John john = new John(paper, pen);

        David david = new David(paper, pen);

        john.start();

        david.start();
    }
}
```

How to avoid dead lock?

```
class Paper {

}

class Pen {

}

class John extends Thread {

    Paper paper;
    Pen pen;
```

```
    John(Paper paper, Pen pen) {

        this.paper = paper;
        this.pen = pen;

    }

    public void run()
    {
      //1. john gets paper
        synchronized (paper) {

            System.out.println("John tries to use(lock) Paper -
Paper available(not locked) - John using Paper - Paper locked");

            System.out.println("John tries to use(lock) Pen - Pen is
available or will be released - John is waiting if Pen is locked");
             //john gets pen
            synchronized (pen) {

                System.out.println("John: Pen locked");
                 //John signing the doc using Paper and Pen
                System.out.println("John: Using Paper and Pen (John
signs the doc)- Work completed");

            }

        }

    }

}

class David extends Thread {

    Paper Paper;
    Pen pen;

    David(Paper Paper, Pen pen) {

        this.Paper = Paper;
        this.pen = pen;

    }

    public void run() {
      //2.David tries to get paper
        synchronized (Paper) {
```

```
            System.out.println("David tries to use(lock) Paper -
Paper is locked by John - David is waiting to get Paper");

            synchronized (pen) {

                System.out.println("David: Paper locked");
                System.out.println("David: Pen locked");
                System.out.println("David: Using Paper and Pen -
Work completed");

            }

        }

    }

}

public class WithoutDeadlockMain {

    public static void main(String[] args) {

        Paper Paper = new Paper();

        Pen pen = new Pen();

        John john = new John(Paper, pen);

        David david = new David(Paper, pen);

        john.start();

        david.start();

    }

}
```